- By default, the controller and action name have no bearing on the URLs or route templates when you use attribute routing. However, you can use the `"[controller]"` and `"[action]"` tokens in your route templates to reduce repetition. They'll be replaced with the current controller and action name.
- The `[HttpPost]` and `[HttpGet]` attributes allow you to choose between actions based on the request's HTTP verb when two actions correspond to the same URL. This is a common pattern in RESTful applications.
- The `[ApiController]` attribute applies several common conventions to your controllers. Controllers decorated with the attribute automatically bind to a request's body instead of using form values, automatically generate a `400 Bad Request` response for invalid requests, and return `ProblemDetails` objects for status code errors. This can dramatically reduce the amount of boilerplate code you must write.
- You can control which of the conventions to apply by using the `ConfigureApiBehaviorOptions()` method and providing a configuration lambda. This is useful if you need to fit your API to an existing specification, for example.
- By default, ASP.NET Core formats the API model returned from a web API controller as JSON. In contrast to legacy ASP.NET, JSON data is serialized using camelCase rather than PascalCase. You should consider this change if you get errors or missing values when using data from your API.